



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
Escuela de Ingeniería
Departamento de Ingeniería Eléctrica
IEE2714 – Fundamentos de Procesamiento de Imágenes
Segundo semestre 2026

Informe - Tarea 2

Filtros Espaciales

Fecha de entrega: 09-10-2026
Máximo Olivares Berríos
molivas@estudiante.uc.cl

Introducción

Este informe contiene los fundamentos del desarrollo para la Tarea 2, asociada a filtros espaciales; enfocándose en cómo la naturaleza del ruido en imágenes afecta las estrategias de restauración. El trabajo explora las limitaciones de los filtros globales, comparándolas con alternativas no lineales y adaptativas; evaluando el impacto de cada parámetros en los algoritmos.

En la primera pregunta de la tarea, se aborda la mitigación del ruido de distribución Poisson sobre una imagen sintética de prueba conformada por regiones homogéneas. Se evalúa inicialmente el desempeño y las limitaciones de un filtrado Gaussiano isotrópico global, para luego contrastarlo con una implementación adaptativa. En esta última, la dispersión del kernel se ajusta dinámicamente en cada píxel mediante la interpolación lineal de los sigmas óptimos encontrados (1.76 para el fondo, 1.39 para el cuadrado central y 1.49 para el círculo de mayor intensidad), a partir de una estimación local de la intensidad. El análisis se centra en la reducción del error cuadrático medio (RMSE) global y por zonas, prestando especial atención al comportamiento del filtro en las fronteras entre regiones.

En la segunda sección, se estudia la preservación de bordes y estructuras complejas mediante la implementación de un esquema de difusión anisotrópica sobre la imagen "camera-man" degradada con ruido Gaussiano. Como punto de partida, se analiza el comportamiento de una aproximación regularizada de Variación Total (TV). Posteriormente, el análisis se expande mediante el diseño de dos coeficientes de difusión alternativos. El primero de ellos se inspira en el modelo exponencial discreto, utilizando la suma de las magnitudes absolutas del gradiente suavizado en los ejes principales. El segundo coeficiente incorpora el Laplaciano local como un indicador de estructura, definiendo el coeficiente de difusión en base a la razón entre el Laplaciano y la magnitud del gradiente suavizado, ponderada por un umbral de contraste. Finalmente, se evalúan las condiciones de estabilidad temporal del esquema, implementando advertencias dinámicas basadas en la cota de Courant-Friedrichs-Lewy:

$$\lambda > 1/4c_{max}$$

Posteriormente se comparan las distintas propuestas en términos de reducción de ruido, protección de bordes y eventual generación de artefactos.

Pregunta 1

Implementación

Para resolver el código solicitado, se crearon tres archivos: `P1.py`, `P1_exp.py` y `P1_graficos.py`. El primero contiene todas las funciones usadas para el código, el segundo se encarga principalmente de la experimentación y el tercero es un *script* diseñado para generar los gráficos de las curvas de desviación estándar (desde ahora, σ) vs RMSE (*Root Mean Square Error*, Raíz del Error Cuadrático Medio), para encontrar la óptima para cada máscara por separado.

Experimentación y análisis

Tras utilizar 8 semillas diferentes durante la experimentación, se llegó a la conclusión de que los valores óptimos (es decir, el que minimiza la RMSE) para cada máscara son:

- $\sigma_{circulo} = 1.49$
- $\sigma_{cuadrado} = 1.39$
- $\sigma_{fondo} = 1.76$
- $\sigma_{global} = 1.54$

Un ejemplo de curva $RMSE(\sigma)$ sería:

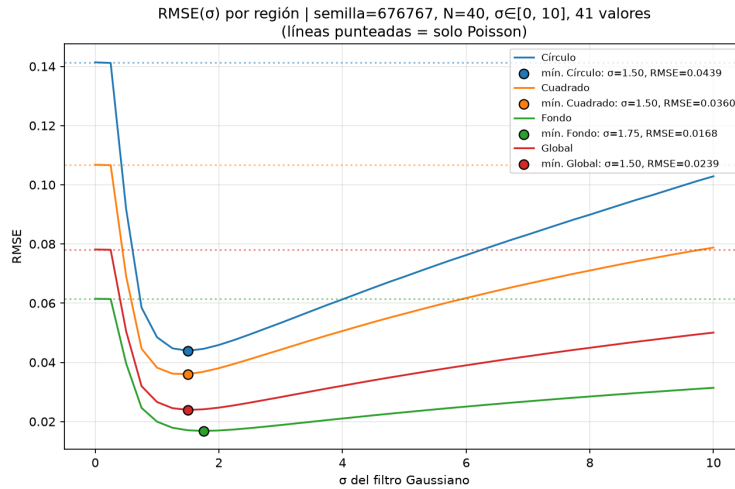


Figura 1: Función $RMSE(\sigma)$ para la semilla 676767.

Y haciendo un zoom entre 0 y 1:

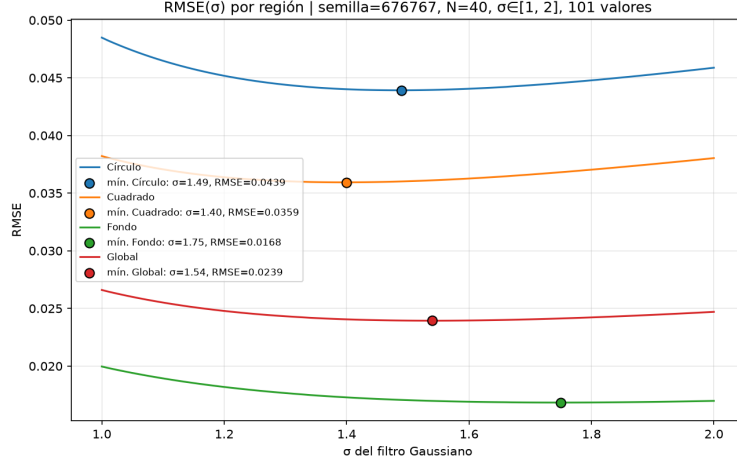


Figura 2: Función $RMSE(\sigma)$ para la semilla 676767 entre 1 y 2.

Como se puede apreciar, para las distintas máscaras, el error se minimiza en distintos valores de σ . Esto es porque a mayor intensidad y a más bordes (de manera independiente) en una máscara, más sube el RMSE; es por esto que, en promedio, el RMSE del fondo suele ser el mayor y el global tiende parecerse mucho al de círculo; porque global actúa como el "promedio" de la imagen.

Para el filtro adaptativo, se crea un banco de imágenes a partir de la imagen con ruido Poisson sin filtrar. Luego, la imagen final es el resultado de la interpolación lineal entre los dos valores de σ que engloben el σ de cada píxel, la operación se hace píxel a píxel. Esta forma de calcular la imagen fue elegida porque reduce el peso computacional, ya que no es necesario calcular kernel distintos para cada píxel y es construido a base de los valores óptimos promedio de σ obtenidos antes. Además, solo depende de la intensidad de los píxeles, no de las máscaras o la ubicación actual; por lo que, en principio, podría funcionar para cualquier imagen. A continuación, la comparación entre el filtro adaptativo (junto a su mapa de valores de σ) y el resultado del filtro aplicado de forma global. También se adjunta la imagen de la diferencia, amplificada 100 veces (todas para la semilla 676767):

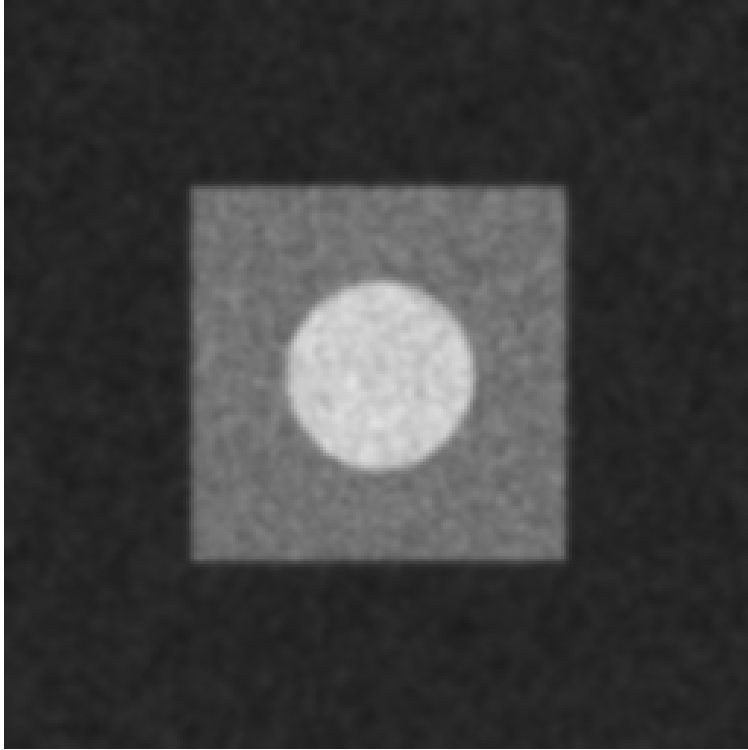


Figura 3: Resultados del filtro adaptativo.

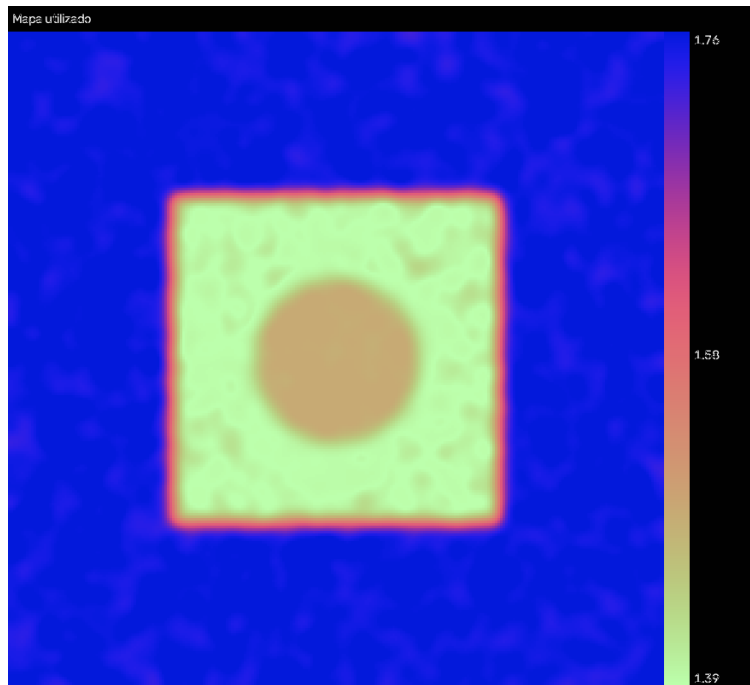


Figura 4: mapa de σ utilizado para calcular la imagen de la figura 3.

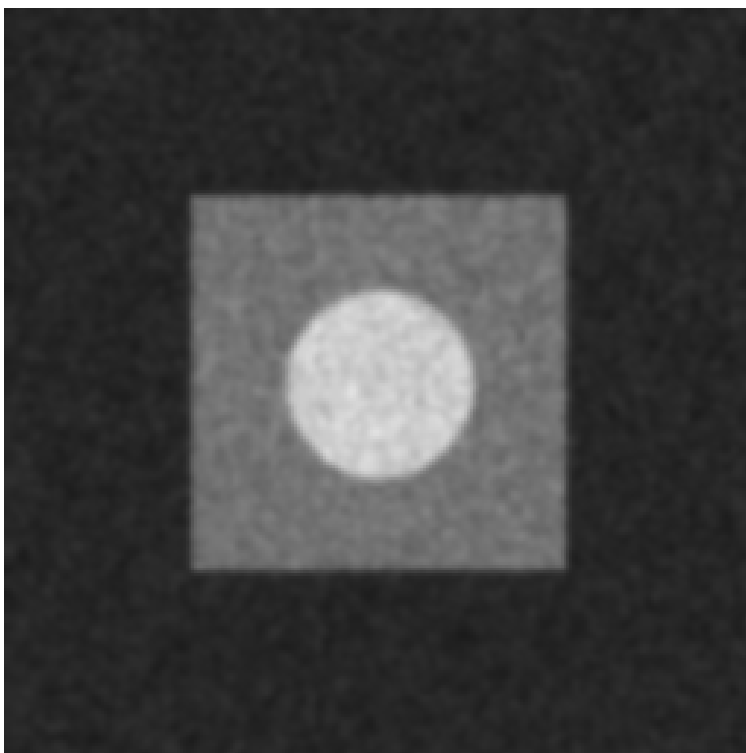


Figura 5: Resultados del filtro global.

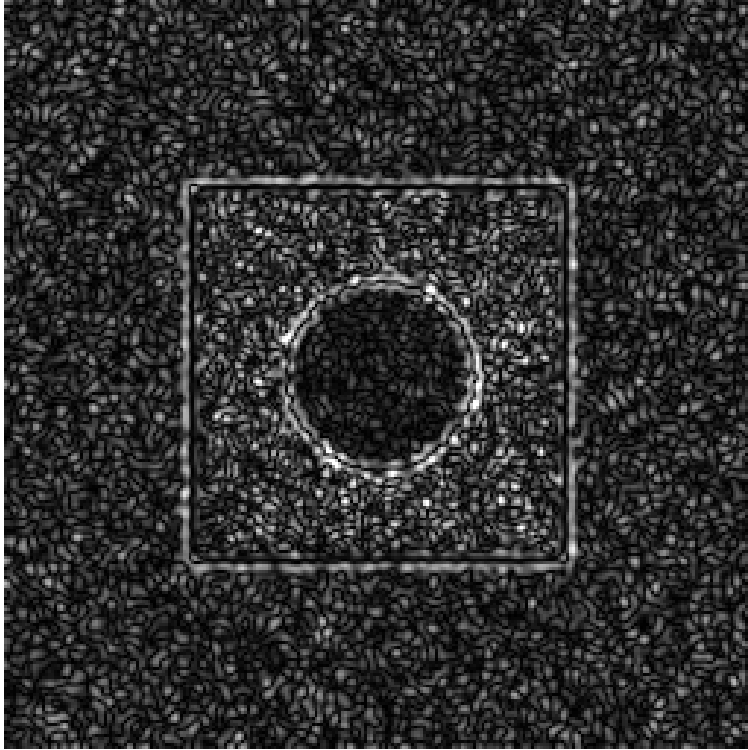


Figura 6: Diferencia entre figura 3 y 5, amplificada 100 veces.

Para concluir con la implementación, es necesario mostrar la diferencia entre el mapa de σ presentado en la figura 4, y el ideal generado por la imagen original:

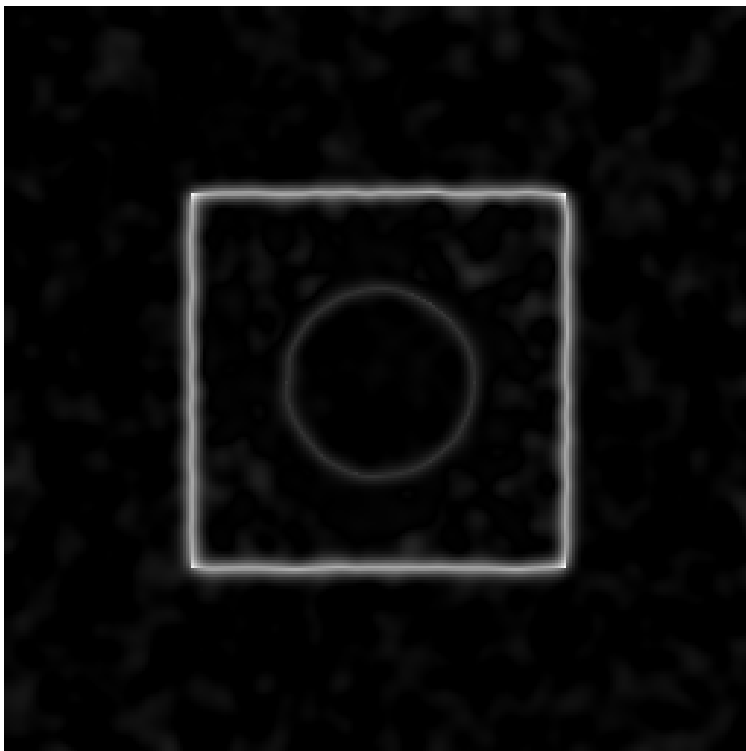


Figura 7: Diferencia entre el mapa de σ usado y el ideal

Como se puede apreciar en la imagen, sí se produce un halo en los bordes de las imágenes. Una solución podría ser reducir el valor de `s` en el código de `P1.text`, pero eso genera imágenes más ruidosas en las zonas de color plano. Otra alternativa es darle algún uso a la mediana en el código.

Preguntas guiadas

¿Por qué la varianza del ruido Poisson depende de la intensidad? Relacione esta propiedad con las diferencias observadas entre las tres curvas $\text{RMSE}(\sigma)$.

El ruido Poisson depende de la intensidad porque, en una distribución de Poisson, la varianza es igual a la media. Para una salida cualquiera:

$$Y \sim \text{Poisson}(N \cdot x)$$

la varianza y media son $N \cdot x$. Y la imagen con ruido poisson que `P1.py` genera sería Y/N , lo que implica $\text{Esperanza} = x$, $\text{Varianza} = x/N$ y, entonces, $\sigma = \sqrt{x/N}$ (con $N = 40$). Es por esto (los distintos valores de x solicitados en el enunciado), que el círculo tiene el mayor ruido, luego el cuadrado y luego el fondo.

¿Por qué se pidió incluir en las máscaras los píxeles próximos a los bordes? ¿Qué ocurriría con el σ óptimo si el RMSE se midiera sólo en una pequeña zona plana muy alejada de cualquier frontera?

Los píxeles de borde son donde el filtro hace daño (porque desenfoca el escalón). Si se excluyeran, el RMSE premiaría suavizar más de lo razonable. En una zona plana lejos de cualquier frontera el sesgo es casi nulo, así que el RMSE seguiría bajando al subir σ , y el mínimo se iría a un σ grande, limitado solo por la distancia al borde más cercano. Ese σ óptimo sería falsamente alto para la imagen completa.

Cómo determina el tamaño del kernel a partir de σ y por qué es necesario normalizar sus coeficientes?

Fue elegido un radio (r) de $3 \cdot \sigma$ y un tamaño total de $(2 \cdot r + 1)$ (el del centro + 2 radios). $3 \cdot \sigma$ porque, en la campana de Gauss, ese radio contiene 99.73 %, y como el kernel utilizado es el producto entre dos vectores idénticos, el área encerrada es de $0.9973^2 = 0.9946$. Se debe normalizar para que el valor de la intensidad no aumente y así no cambie el valor de la RMSE.

Explique exactamente cómo obtiene $\hat{\mu}(x,y)$ y cómo transforma ese valor en $\sigma(x,y)$.
 Seleccione un píxel de cada una de las tres regiones e indique $\hat{\mu}$, σ y el kernel aplicado

Pregunta 2

Implementación

Tal como la pregunta anterior, la construcción de esta pregunta circula principalmente alrededor de funciones y luego ser llamadas en una función mayor. Para la experimentación, se implementaron las tres funciones solicitadas:

$$c_{vt} = \frac{1}{\sqrt{|\nabla u|^2 + \epsilon^2}}$$

$$c_1 = \exp[-(\frac{|u_x| + |u_y|}{k})^2]$$

$$c_2 = \frac{1}{1 + \frac{((1-b) \cdot (\nabla u))^2}{k}} \{b := \text{clip}(\frac{\Delta u}{\nabla u}, 0, 1)\}$$

c_1 fue elegido de esa forma, ya que al tener una ecuación que opera la distancia de una manera diferente, puede traer resultados que se comporten diferente. En particular en torno a las diagonales. Mientras que c_2 fue elegido de esta forma al utilizar el Laplaciano como un indicador de cambio para el parámetro b cumple la condición de uso del Laplaciano, y la razón del Laplaciano y el gradiente logran que se protejan los bordes, versus que una razón grande fuerza a que se difumine; lo que calza con el comportamiento del Laplaciano. El criterio de estabilidad que fue elegido es el de condición de Courant-Friedrichs-Lewy (CFL), dicha condición consiste en que un píxel no puede transmitir más brillo del que posee (ese

es c_{max}), y lo que se transfiere (c), se divide en 4 partes. Es decir, todo lo que se transfiere en un intervalo de tiempo, y lo que puede transmitirse por dirección puede modelarse como:

$$\lambda \cdot c_{max} \leq \frac{1}{4}$$

que corresponde con el criterio CFL de estabilidad.

Experimentación y Análisis

A continuación, una figura que muestra distintos resultados para los mapas creados por distintos valores de ϵ (en particular, 0.01, 0.2, 1, 3) para un paso temporal (de ahora en adelante, λ) de 0.02 y 10 repeticiones del algoritmo:

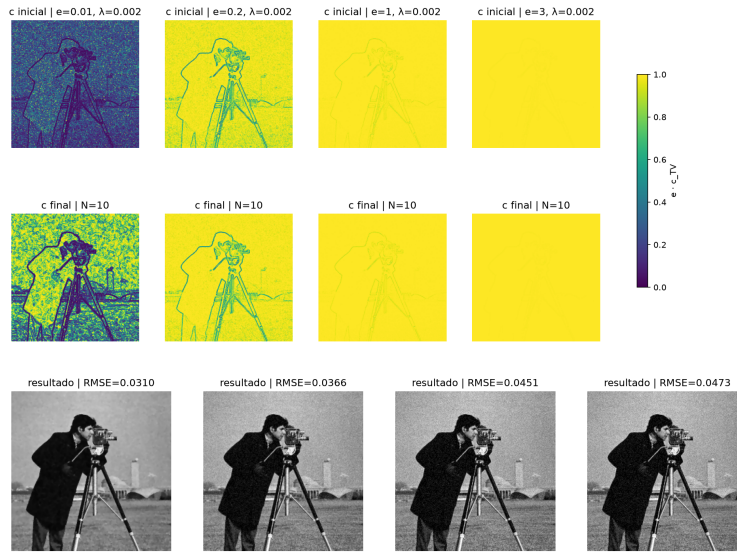


Figura 8: Resultados a distintas iteraciones para ϵ .

Como puede apreciarse, existe un "punto dulce" para el valor de ϵ para cada configuración, y lo mismo puede decirse de los tres valores con los que se itera (N, λ, ϵ). En particular, para $N = 10$ y $\lambda = \lambda_{max}$, parece estar entre 0.01 y 0.2. También implica que es posible hacer el algoritmo muchas veces o con "demasiada fuerza" se vuelva solamente ruido (similar al ruido de sal y pimienta).

Para el segundo experimento, a continuación están las imágenes resultados.

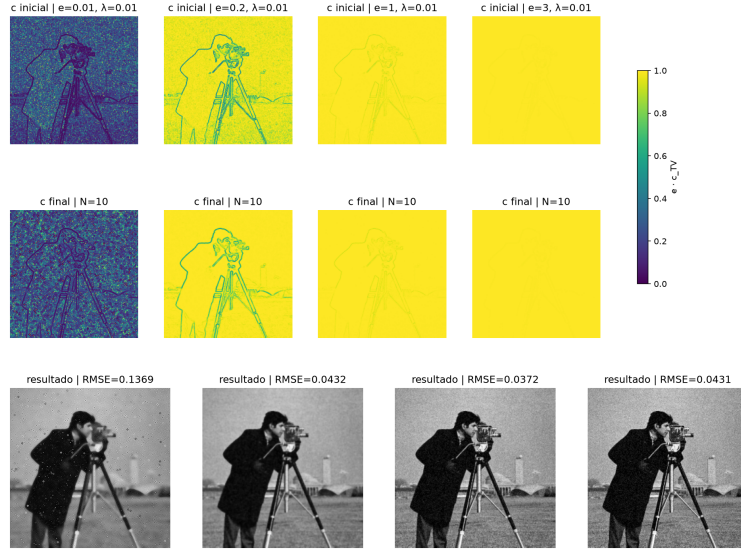


Figura 9: Resultados a distintas iteraciones para ϵ , $\lambda = 0.01$.

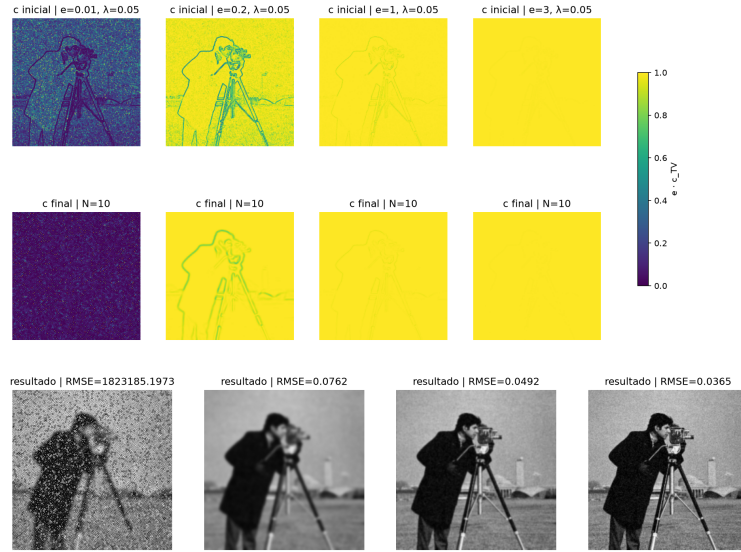


Figura 10: Resultados a distintas iteraciones para ϵ , $\lambda = 0.05$.

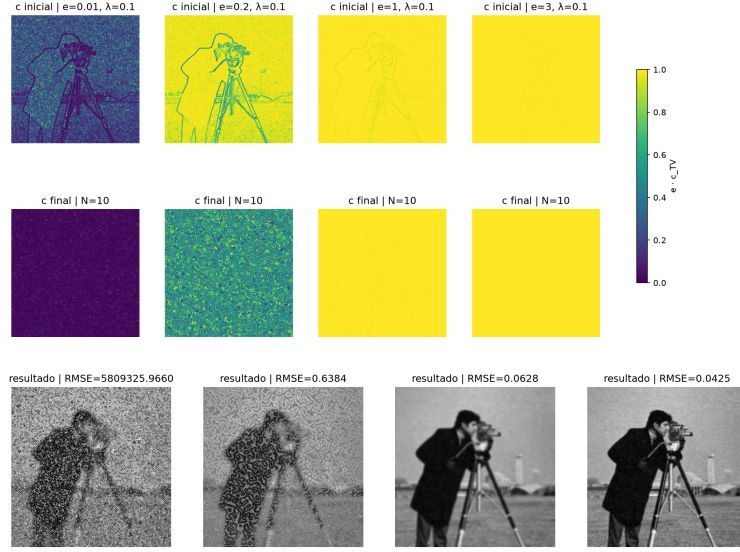


Figura 11: Resultados a distintas iteraciones para ϵ , $\lambda = 0.1$.

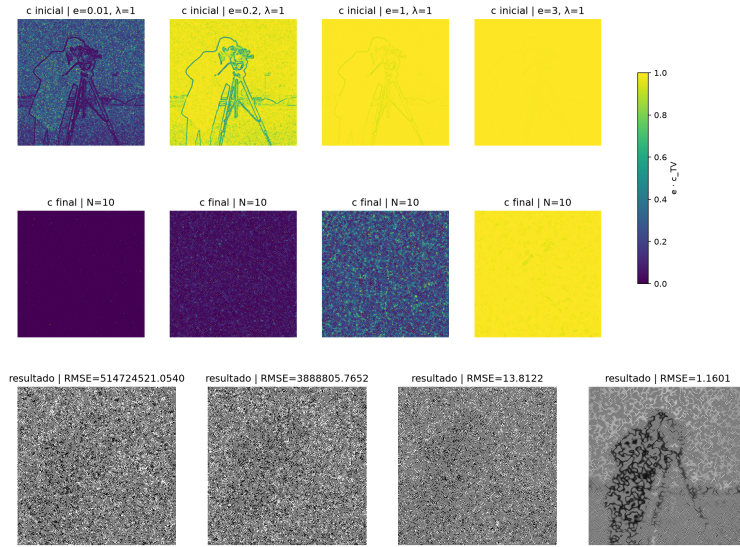


Figura 12: Resultados a distintas iteraciones para ϵ , $\lambda = 1$.

Podemos concluir que cuando una imagen se vuelve ruido, su RMSE cambia lentamente entre valores, como puede apreciarse en los pares de $\lambda = 1$ y $\lambda = 10$, pero su mapa cambia con fuerza

Cuando hacemos la misma iteración en los coeficientes propuestos c_1 y c_2 , obtenemos las siguientes imágenes:

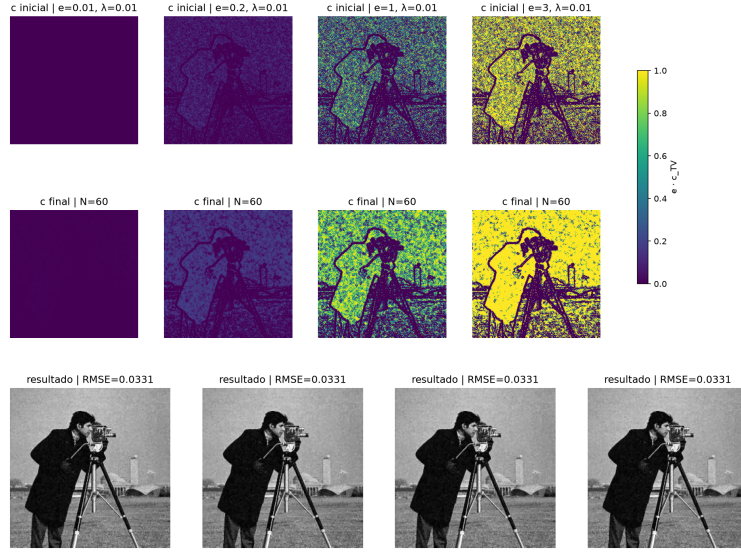


Figura 13: Resultados a distintas iteraciones para $c_1, \epsilon, \lambda = 0.01$.

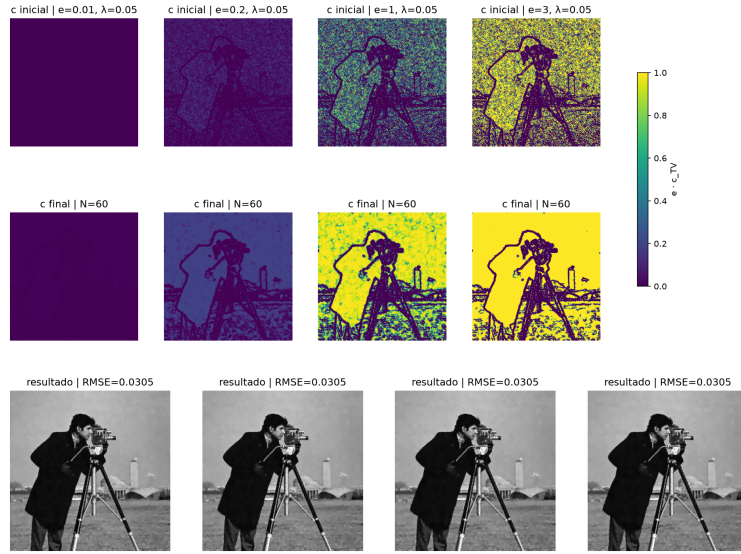


Figura 14: Resultados a distintas iteraciones para $c_1, \epsilon, \lambda = 0.05$.

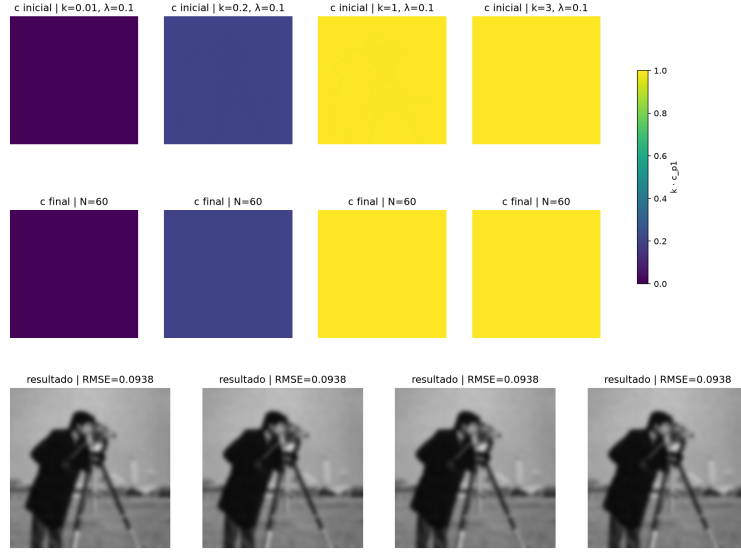


Figura 15: Resultados a distintas iteraciones para $c_1, \epsilon, \lambda = 0.1$.

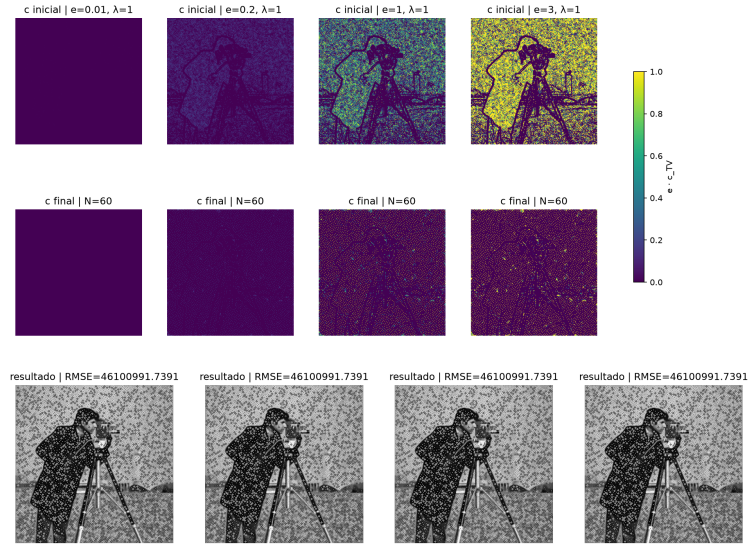


Figura 16: Resultados a distintas iteraciones para $c_1, \epsilon, \lambda = 1$.

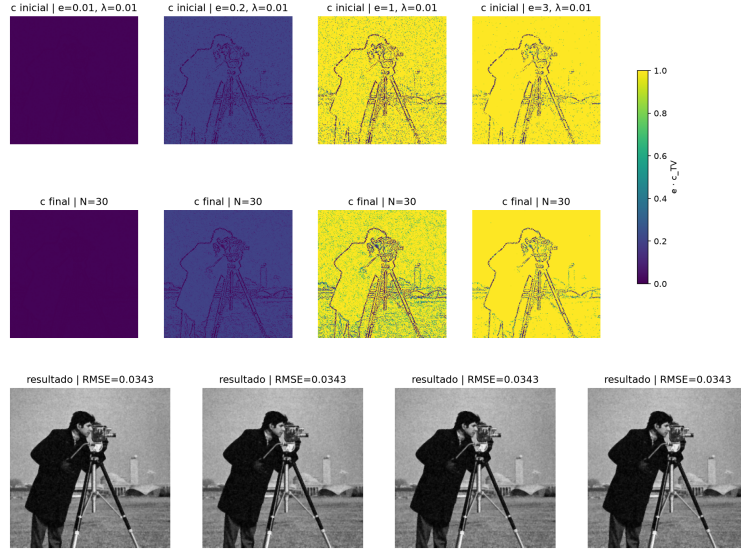


Figura 17: Resultados a distintas iteraciones para $c_2, \epsilon, \lambda = 0.01$.

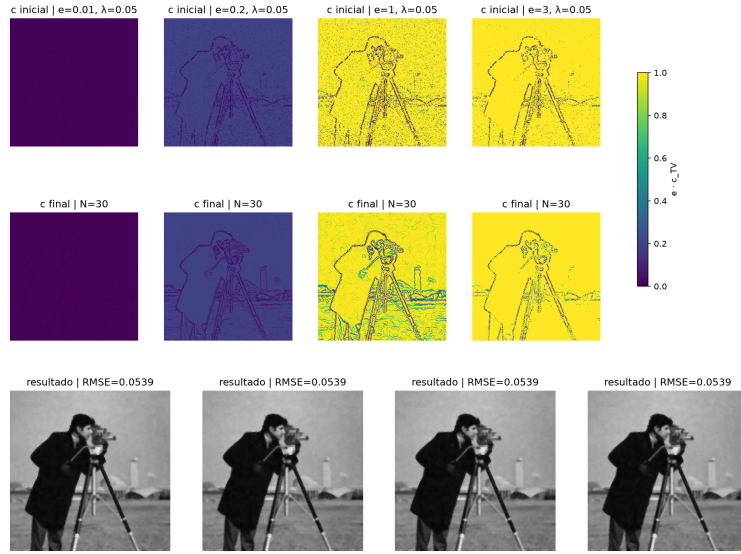


Figura 18: Resultados a distintas iteraciones para $c_2, \epsilon, \lambda = 0.05$.

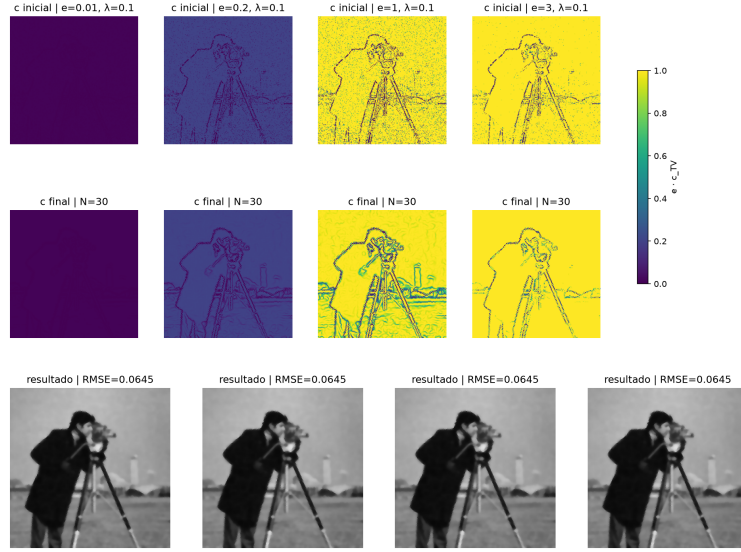


Figura 19: Resultados a distintas iteraciones para $c_2, \epsilon, \lambda = 0.1$.

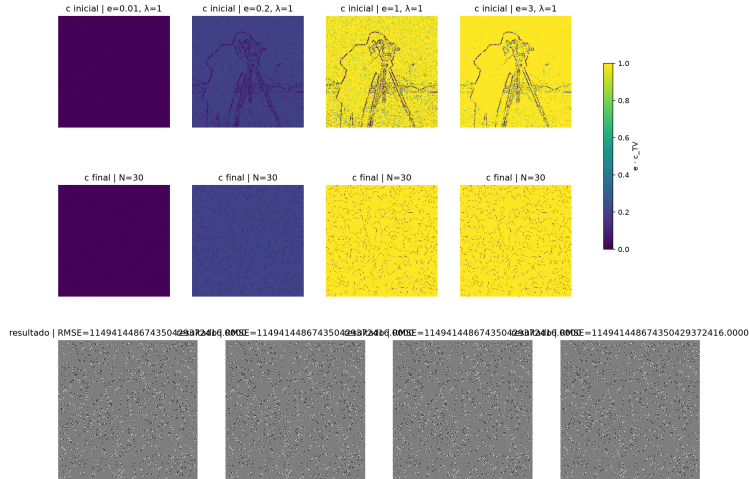


Figura 20: Resultados a distintas iteraciones para $c_1/2, \epsilon, \lambda = 1$.

Como puede verse, cada función tiene un valor de N que la hace funcionar mejor, porque utilizan modelos diferentes.

Mi propuesta c_2 utiliza Laplaciano como indicador de ruido. Se usa sin normalizar. Como se mostró antes, es extremadamente sensible dadas las condiciones correctas, por eso debe ser usado con el ponderador b

Como puede verse en las siguientes figuras:



Figura 21: Resultado cVT, $RMSE = 0.04391$.



Figura 22: Resultado c_1 , $RMSE = 0.04947$.



Figura 23: Resultado c_2 , $RMSE = 0.04585$.

$\lambda = \text{fracción} \cdot \lambda_{max}$, con fracciones 0.1, 0.5, 0.9 y 1.4 (usadas en la experimentación). Las tres primeras están dentro de la cota y la última la supera a propósito, para observar qué ocurre al pasarla. Se usó el c_{max} teórico en vez de calcularlo de la imagen, porque el c_{max} real en cada iteración es menor o igual.

Preguntas guiadas

En qué momento de cada iteración se calculan el gradiente, el Laplaciano y c ?
Indique qué versión de la imagen utiliza cada cálculo y dónde se encuentra implementado en su código

Se calculan los tres valores en cada iteración al inicio, una vez la nueva imagen actual del bucle es asignada, la cual se asigna al final de haber aplicado todas las transformaciones matemáticas

En el caso TV, explique el papel de ϵ . ¿Qué valor toma aproximadamente cTV cuando $|\nabla u|$ es muy pequeño? ¿Cómo cambia esa respuesta al disminuir ϵ y qué consecuencia tiene para la estabilidad del esquema explícito?

Tiende a ϵ . Si disminuimos ϵ , estamos multiplicando por una fracción de tipo $\frac{1}{\epsilon}$, que es un valor mayor que uno, lo que destruye su estabilidad al no converger a nada

Qué comportamiento obtiene si c se reemplaza por una constante espacial y temporal? Relacione este caso con la difusión isotrópica y el suavizado Gaussiano.

Tiende a un suavizado Gaussiano, porque hacer la convolución tantas veces equivale a ello, al ser lo mismo que hacer N sumas aleatorias.

Anexos:

- Enlace al repositorio: https://github.com/Oikkoi/T2_Imagenes.git